

智慧银行实验教程

实验四：ESG 银行评级查询与对比分析系统开发

何彦霖 金融 202301 202301746

2026 年 7 月 1 日

1 实验目的

本次实验旨在使用 BMAD (Breakthrough Method of Agile AI-Driven Development) 方法开发一个完整的 ESG (环境、社会、治理) 银行评级查询与对比分析系统，掌握全栈 Web 开发技术和数据可视化方法。具体学习目标包括：

- 掌握 BMAD 敏捷 AI 驱动开发方法
- 深入理解 ESG 评级体系和评估维度
- 掌握 Flask 后端 API 开发方法
- 学习 RESTful API 设计原则
- 掌握 ECharts 数据可视化技术
- 实现多维度对比分析功能
- 开发行业统计和洞察分析功能
- 掌握 Vercel 部署和 ngrok 外网暴露方法
- 学习项目同步云端的操作流程

2 实验环境

项目	配置
操作系统	Windows 10/11
Python 版本	3.12
后端框架	Flask 3.0.0+
前端技术	HTML5 + CSS3 + JavaScript (ES6+)
图表库	ECharts 5.4.0
数据处理	Pandas 2.2.0+
部署工具	Vercel CLI, ngrok
开发方法	BMAD
Git 仓库	https://cnb.cool/yanlinsicau/first
创建时间	2026-06-25
Git 提交	ce7db46

表 1: 实验环境配置

3 BMAD 开发方法概述

BMAD（Breakthrough Method of Agile AI-Driven Development）是一种敏捷 AI 驱动的开发方法，包含四个核心阶段：

阶段	名称	核心内容
Analysis	分析阶段	需求分析、目标用户定义、功能需求梳理
Planning	规划阶段	PRD 文档编写、用户故事拆分、API 设计
Solution	方案阶段	架构设计、技术选型、目录结构规划
Execution	实施阶段	代码开发、测试验证、部署上线

表 2: BMAD 方法四阶段

4 实验步骤

4.1 步骤 1：BMAD 分析阶段 - 需求分析

4.1.1 项目背景

ESG（环境 Environmental、社会 Social、治理 Governance）评级已成为评估企业可持续发展能力的重要指标。银行业作为金融体系的核心，其 ESG 表现直接影响投资者决策和监管合规。

4.1.2 目标用户

用户角色	核心需求
银行客户经理	查询客户 ESG 评级，辅助信贷决策
投资分析师	对比分析银行 ESG 表现，支持投资建议
合规部门	监控银行 ESG 合规情况
研究人员	进行银行业 ESG 趋势分析

表 3: 目标用户及其需求

4.1.3 功能需求优先级

功能模块	描述	优先级
ESG 数据查询	查询指定银行的 ESG 三维度评分	P0
多银行对比	选择多个银行进行 ESG 对比分析	P0
筛选功能	按银行名称、行业、评级范围筛选	P1
可视化展示	ECharts 图表展示评分和对比	P0
行业统计	各行业平均 ESG 评分统计	P1

表 4: 功能需求优先级

4.1.4 数据需求

- 环境维度 (E): 碳排放、绿色信贷占比、环保投入、节能减排措施
- 社会维度 (S): 员工权益、客户服务、社区贡献、社会责任履行
- 治理维度 (G): 董事会结构、风险管理、合规体系、信息披露透明度

详细文档见: analysis.md

4.2 步骤 2：BMAD 规划阶段 - PRD 文档与 API 设计

4.2.1 用户故事

ID	用户故事	验收标准
US-001	作为银行客户经理，我想要查询指定银行的 ESG 三维度评分	输入银行名称可查询 ESG
US-002	作为投资分析师，我想要选择多个银行进行 ESG 对比分析	可选择 2-5 个银行进行对比
US-003	作为研究人员，我想要按行业或评级范围筛选银行	支持按行业类型筛选；支持

表 5: 用户故事清单

4.2.2 API 接口设计

接口	方法	说明
/api/banks	GET	获取所有银行列表
/api/bank/<id>	GET	根据 ID 获取银行详情
/api/bank/search	GET	根据名称搜索银行
/api/banks/compare	GET	多银行对比
/api/banks/filter	GET	筛选银行
/api/statistics/industry	GET	行业统计
/api/industries	GET	行业类型列表
/api/ratings	GET	评级等级列表

表 6: API 接口清单

详细文档见：prd.md

4.3 步骤 3：BMAD 方案阶段 - 架构设计

4.3.1 系统架构

前端层

查询页面

对比页面

筛选页面

ECharts图表

HTTP/JSON

后端层

Flask API
 /api/banks
 /api/bank/<id>
 /api/compare
数据服务层

数据层
 模拟ESG数据 (Python Dict)

4.3.2 技术栈

层级	技术
后端框架	Flask 2.3+
数据处理	Python 3.8+
前端渲染	HTML5/CSS3
可视化	ECharts 5.4+
HTTP 交互	Fetch API

表 7: 技术栈选择

4.3.3 目录结构

esg-bank-tool/
 app.py # Flask主应用
 data.py # ESG模拟数据
 requirements.txt # Python依赖
 vercel.json # Vercel部署配置
 templates/
 index.html # 主页面模板
 static/
 css/
 style.css # 样式文件
 js/
 main.js # 前端交互逻辑
 analysis.md # 需求分析文档
 prd.md # PRD文档
 architecture.md # 架构设计文档

README.md # 项目说明

详细文档见: architecture.md

4.4 步骤 4: BMAD 实施阶段 - 代码开发

4.4.1 项目初始化与依赖安装

创建 requirements.txt:

```
flask>=3.0.0
pandas>=2.2.0
numpy>=1.24.0
pyngrok>=8.0.0
```

安装依赖:

```
pip install -r requirements.txt
```

4.4.2 ESG 数据构建

数据包含 68 家银行的 ESG 评分:

银行类型	代表银行	数量
国有大型银行	工商银行、建设银行、中国银行	6 家
全国性股份制银行	招商银行、兴业银行、浦发银行	12 家
城市商业银行	北京银行、上海银行、南京银行	30 家
农村商业银行	重庆农商行、北京农商行	20 家
合计		68 家

表 8: 银行分类统计

数据结构示例:

```
BANK_ESG_DATA = [
    {
        "id": 1,
        "name": "工商银行",
        "industry": "国有大型银行",
        "e_score": 82.5,
        "s_score": 85.3,
        "g_score": 88.7,
        "total_score": 85.5,
```

```
    "rating": "AAA",
    "e_detail": {
        "carbon_emission": 45.2,
        "green_credit_ratio": 18.5,
        "eco_investment": 120.5
    },
    "s_detail": {
        "employee_satisfaction": 87.5,
        "customer_complaint_rate": 0.12,
        "community_contribution": 85.0
    },
    "g_detail": {
        "board_independence": 45.0,
        "risk_management": 92.5,
        "info_transparency": 95.0
    }
}
# ... 其他67家银行数据
]
```

4.4.3 评级体系

评级等级	总分范围
AAA	90-100
AA	80-89
A	70-79
BBB	60-69
BB	50-59
B	0-49

表 9: ESG 评级标准

4.4.4 Flask 后端 API 开发

```
from flask import Flask, jsonify, request
from data import (
    get_all_banks, get_bank_by_id, get_bank_by_name,
    get_banks_by_ids, filter_banks, get_industry_statistics,
    INDUSTRIES, RATINGS
```

```

)

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/api/banks', methods=['GET'])
def api_get_banks():
    banks = get_all_banks()
    return jsonify({
        "success": True,
        "data": {"banks": banks, "count": len(banks)}
    })

@app.route('/api/bank/<int:bank_id>', methods=['GET'])
def api_get_bank(bank_id):
    bank = get_bank_by_id(bank_id)
    if bank:
        return jsonify({"success": True, "data": bank})
    return jsonify({"success": False, "error": "银行不存在"}), 404

```

4.4.5 对比分析 API

```

@app.route('/api/banks/compare', methods=['GET'])
def api_compare_banks():
    ids_str = request.args.get('ids', '')
    ids = [int(id.strip()) for id in ids_str.split(',')]

    banks = get_banks_by_ids(ids)

    avg_e = sum(b["e_score"] for b in banks) / len(banks)
    avg_s = sum(b["s_score"] for b in banks) / len(banks)
    avg_g = sum(b["g_score"] for b in banks) / len(banks)

    comparison = {
        "banks": banks,
        "avg_scores": {"e": round(avg_e, 2), "s": round(avg_s, 2), "g": round(avg_g, 2)},
    }

```



```
    "deviation": [  
      {"bank": b["name"], "e": round(b["e_score"] - avg_e, 2),  
      ...}  
      for b in banks  
    ]  
  }  
  return jsonify({"success": True, "data": comparison})
```

4.4.6 前端页面开发

前端页面包含四个核心模块：

模块	功能
评级查询	搜索银行、查看 ESG 详情、雷达图和仪表盘图展示
对比分析	选择 2-5 家银行对比、多维度图表展示、偏差分析
筛选查询	按行业/评级/评分范围筛选、筛选结果展示
行业统计	行业平均分统计、排名榜单、关键洞察

表 10: 前端模块功能

4.4.7 前端交互逻辑

```
async function searchBanks() {  
  const keyword = document.getElementById('search-input').value;  
  const response = await fetch(`/api/bank/search?name=${keyword}`);  
  const result = await response.json();  
  if (result.success) {  
    renderBankList(result.data.banks);  
  }  
}  
  
async function startCompare() {  
  const selectedIds = getSelectedBankIds();  
  const response = await fetch(`/api/banks/compare?ids=${  
    selectedIds.join(',')`);  
  const result = await response.json();  
  if (result.success) {  
    renderComparisonCharts(result.data);  
  }  
}
```

4.4.8 部署配置

创建 `vercel.json`:

```
{
  "builds": [
    {
      "src": "app.py",
      "use": "@vercel/python"
    }
  ],
  "routes": [
    {
      "src": "/*",
      "dest": "app.py"
    }
  ]
}
```

添加 ngrok 外网暴露功能:

```
if __name__ == '__main__':
    try:
        from pyngrok import ngrok
        http_tunnel = ngrok.connect(5000)
        print(f"外网访问地址: {http_tunnel.public_url}")
    except Exception as e:
        print(f"ngrok启动失败: {e}")

app.run(debug=True, host='0.0.0.0', port=5000)
```

4.5 步骤 5: 同步云端

```
# 初始化Git仓库
cd 小组作业/esg-bank-tool
git init

# 添加文件
git add .

# 提交
```

```
git commit -m "feat: add ESG bank rating tool with multi-dimension analysis"

# 同步到云端仓库
git remote add origin https://cnb.cool/yanlinsicau/first.git
git push -u origin main
```

5 实验结果

5.1 功能实现情况

功能模块	状态	说明
评级查询	已完成	支持搜索和详情展示，含雷达图、仪表盘图
对比分析	已完成	支持 2-5 家银行多维度对比
筛选查询	已完成	支持行业、评级、评分范围筛选
行业统计	已完成	包含概览、明细、分布、排名四大面板
多维度对比	已完成	6 个维度切换（概览/E/S/G/偏差/矩阵）
偏差分析	已完成	自动计算偏差和生成洞察
矩阵分布	已完成	E-S 散点图和饼图
关键洞察	已完成	自动生成分析结论

表 11: 功能实现状态

5.2 数据规模

指标	数值
银行总数	68 家
数据维度	$E(3 \text{ 项}) + S(3 \text{ 项}) + G(3 \text{ 项}) = 9 \text{ 项}$
评级等级	6 个等级 (AAA, AA, A, BBB, BB, B)
银行类型	4 大类
API 端点	8 个
图表类型	10 种 +

表 12: 数据规模统计

5.3 Git 提交记录

ce7db46 feat: add ESG bank rating tool with multi-dimension analysis

新增文件:

- 小组作业/esg-bank-tool/app.py (后端API)
- 小组作业/esg-bank-tool/data.py (ESG数据)
- 小组作业/esg-bank-tool/static/css/style.css (样式)
- 小组作业/esg-bank-tool/static/js/main.js (前端逻辑)
- 小组作业/esg-bank-tool/templates/index.html (HTML页面)
- 小组作业/esg-bank-tool/requirements.txt (依赖)
- 小组作业/esg-bank-tool/vercel.json (部署配置)
- 小组作业/esg-bank-tool/analysis.md (需求分析)
- 小组作业/esg-bank-tool/prd.md (PRD文档)
- 小组作业/esg-bank-tool/architecture.md (架构设计)
- 小组作业/esg-bank-tool/README.md (项目说明)

统计:

- 11个文件
- 4713行代码
- 68家银行数据
- 8个API端点

6 问题与解决

1. 问题: ngrok 认证失败

解决: 需要注册 ngrok 账号获取 authtoken, 使用 `ngrok config add-authtoken YOUR-Token` 配置

2. 问题: Vercel 登录失败 (环境变量问题)

解决: 检查网络连接, 使用备用登录方式

3. 问题: 前端图表容器尺寸自适应问题

解决: 实现 `window.resize` 事件监听, 所有图表重新 `resize`

4. 问题: 数据归一化处理

解决: 实现 `min-max` 归一化函数, 将不同量纲指标转换到 0-100 范围

5. 问题: 偏差分析计算精度

解决: 使用 `round()` 函数保留两位小数, 确保数据一致性

6. 问题: 图表中文乱码

解决: 检查 HTML 字符编码, 统一使用 UTF-8 编码

7. 问题： CORS 跨域问题
- 解决：在 Flask 中添加 CORS 支持，设置允许的源

7 四次实验总结

7.1 实验内容回顾

实验	主题	核心内容
实验一	基础环境搭建与数据处理	Python/Node.js 安装、HTML 贪吃蛇游戏、AI 论文解读助
实验二	CNB CLI 与技能管理	CNB CLI 安装、Skills 工具配置、PPT 定价报告生成、文
实验三	OpenBB 金融分析与系统开发	OpenBB Finance 技能安装、SpaceX 投资分析、BMAD 技
实验四	ESG 银行评级系统开发	BMAD 方法开发、ESG 评级工具、全栈 Web 开发、数据

表 13: 四次实验内容总结

7.2 技术能力提升

技术领域	掌握程度
Python 开发	熟练
Flask 后端开发	熟练
前端开发 (HTML/CSS/JS)	熟练
ECharts 数据可视化	熟练
Git 版本控制	熟练
Vercel 部署	掌握
BMAD 开发方法	掌握
ESG 评级体系	了解

表 14: 技术能力提升总结

7.3 项目成果统计

指标	数值
完成实验次数	4 次
开发项目数	5 个
生成文档数	6 份实验报告
代码总行数	10000+ 行
Git 提交次数	多次
云端同步项目数	4 个

表 15: 项目成果统计

8 实验心得

通过本次实验，我系统学习了 BMAD 敏捷 AI 驱动开发方法，并完成了 ESG 银行评级系统的全栈开发，取得了以下收获：

- 掌握了 BMAD 方法的四个核心阶段（分析、规划、方案、实施）
- 掌握了 Flask 框架的 RESTful API 开发方法
- 熟悉了 ECharts 图表库的高级用法
- 理解了 ESG 评级体系的结构和指标设计
- 学会了前后端分离开发模式
- 掌握了数据可视化最佳实践
- 提高了项目架构设计和代码组织能力
- 学会了 Vercel 和 ngrok 部署方法
- 掌握了项目同步云端的操作流程

同时也认识到一些不足：

- 数据为模拟数据，缺乏真实数据源支撑
- 用户认证和权限管理尚未实现
- 缺乏单元测试和自动化测试
- 前端界面可以进一步优化

- 性能优化空间较大（大数据量场景）

未来改进方向：接入真实 ESG 数据源，实现用户认证和权限管理，增加数据缓存机制，优化大数据量查询性能，开发移动端应用，添加数据导出功能（PDF/Excel）。